

Devoir surveillé terminal (blanc)

Durée 2h00
(Corrections)

Exercice 1 (Questions de cours - 2 pts)

1°) Qu'est-ce qu'une grammaire contextuelle ? Donnez un exemple.

Solution : Une grammaire $G = (T, N, R, S)$ est contextuelle si ses règles sont de la forme suivante :
 $uAv \rightarrow uvw$ où $A \in N$, $u, v \in (T \cup N)^*$, $w \in (T \cup N)^+$

La grammaire suivante est contextuelle :

$$\begin{array}{ll} S & \rightarrow aSb \\ aS & \rightarrow aaAb \\ A & \rightarrow \epsilon \end{array}$$

2°) Qu'est-ce que la génération de code intermédiaire ? Pourquoi est-elle importante ?

Solution : Le principe de la génération de code intermédiaire est de générer un code assembleur qui est indépendant de la machine, par exemple à l'aide de quadruplets. L'avantage est d'apporter des techniques d'optimisation sur le code, indépendamment de la machine. Elles peuvent donc être factorisées quelle que soit l'architecture cible et seule la transformation de ce code intermédiaire en assembleur est dépendante de la machine.

Exercice 2 (Analyseur LL(1) - 5 pts)

Soit la grammaire G suivante :

$$\begin{array}{ll} S & \rightarrow (A \quad (1) \\ A & \rightarrow B) \quad (2) \\ A & \rightarrow) \quad (3) \\ B & \rightarrow \text{entier } C \quad (4) \\ B & \rightarrow S C \quad (5) \\ C & \rightarrow , B \quad (6) \\ C & \rightarrow \epsilon \quad (7) \end{array}$$

1°) Donnez la définition formelle de cette grammaire.

Solution : (1 pt) Soit la grammaire $G = (T, N, R, S)$ avec :

- $T = \{\text{entier}, '(', ')', ', ' \}$
- $N = \{S, A, B, C\}$
- $S = S$
- $R = \{ S \rightarrow (A, A \rightarrow B), A \rightarrow), B \rightarrow \text{entier } C, B \rightarrow S C, C \rightarrow , B, C \rightarrow \epsilon \}$

2°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

Solution : (2 pts)

- $\text{Premier}(S) = \{ '(' \}$
- $\text{Premier}(A) = \{ '(', \text{entier}, ')' \}$
- $\text{Premier}(B) = \{ '(', \text{entier} \}$
- $\text{Premier}(C) = \{ \epsilon, ', ' \}$

- $Suivant(S) = \{ \epsilon, ', ', ' \}$
- $Suivant(A) = \{ \epsilon, ', ', ' \}$
- $Suivant(B) = \{ ' \}$
- $Suivant(C) = \{ ' \}$

3°) Construisez la T ASD.

Solution : (2 pts) Voici la table obtenue :

	(	)	,	entier	¬
S	$S \rightarrow (A$				
A	$A \rightarrow B)$	$A \rightarrow)$		$A \rightarrow B)$	
B	$B \rightarrow S C$			$B \rightarrow entier C$	
C		AVC	$C \rightarrow, B$		
(	$AVCTS$				
)		$AVCTS$			
,			$AVCTS$		
entier				$AVCTS$	
¬					ACC

Exercice 3 (Analyseur SLR - 9 pts)

Soit la grammaire suivante :

- $$\begin{aligned}
 L &\rightarrow (A) & (1) \\
 L &\rightarrow () & (2) \\
 A &\rightarrow A , entier & (3) \\
 A &\rightarrow A , L & (4) \\
 A &\rightarrow entier & (5) \\
 A &\rightarrow L & (6)
 \end{aligned}$$

1°) Donnez une dérivation pour le texte (1,(1,2),())

Solution : (1 pt) $L \vdash (A) \vdash (A,L) \vdash (A,()) \vdash (A,L,()) \vdash (A,(A),()) \vdash (entier, (A), ()) \vdash (entier, (A,entier), ()) \vdash (entier, (entier, entier), ())$

2°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

Solution : (1 pt)

- $Premier(L) = \{ ' (' \}$
- $Premier(A) = \{ entier, ' (' \}$
- $Suivant(L) = \{ \epsilon, ')', ', ' \}$
- $Suivant(A) = \{ ')', ', ' \}$

3°) Construisez les ensembles d'items LR(0).

Solution : (3 pts) On ajoute la règle $L' \rightarrow L \neg$

- $I_0 = Fermeture(L' \rightarrow .L \neg) = \{ L' \rightarrow .L \neg, L \rightarrow .(A), L \rightarrow .() \}$
- $I_1 = Transition(I_0, L) = \{ L' \rightarrow L. \neg \}$
- $I_2 = Transition(I_0, () = \{ L \rightarrow (.A), L \rightarrow (.), A \rightarrow .A, entier, A \rightarrow .A, L, A \rightarrow .entier, A \rightarrow .L, L \rightarrow .(A), L \rightarrow .() \}$
- $I_3 = Transition(I_2, L) = \{ A \rightarrow L. \}$
- $I_4 = Transition(I_2, A) = \{ L \rightarrow (A.), A \rightarrow A., entier, A \rightarrow A., L \}$
- $Transition(I_2, () = I_2$

- $I_5 = \text{Transition}(I_2,) = \{ L \rightarrow (). \}$
- $I_6 = \text{Transition}(I_2, \text{entier}) = \{ A \rightarrow \text{entier}. \}$
- $I_7 = \text{Transition}(I_4,) = \{ L \rightarrow (A). \}$
- $I_8 = \text{Transition}(I_4, ,) = \{ A \rightarrow A, \text{entier}, A \rightarrow A, .L, L \rightarrow .(A), L \rightarrow .() \}$
- $I_9 = \text{Transition}(I_8, L) = \{ A \rightarrow A, L. \}$
- $\text{Transition}(I_8, () = I_2$
- $I_{10} = \text{Transition}(I_8, \text{entier}) = \{ A \rightarrow A, \text{entier}. \}$

4°) Construisez la table d'analyse selon la méthode SLR.

Solution : (2 pts)

États	Action					Successeur	
	(	)	,	entier	⊢	L	A
0	d,2					1	
1					accepter		
2	d,2	d,5		d,6		3	4
3		r,6	r,6				
4		d,7	d,8				
5		r,2	r,2		r,2		
6		r,5	r,5				
7		r,1	r,1		r,1		
8	d,2			d,10		9	
9		r,4	r,4				
10		r,3	r,3				

5°) Analysez le mot ((1),(1,3),2)

Solution : (2 pts) Voici l'exécution :

Pile	Entrée	Action
0	((1),(1,3),2)⊢	d,2
0(2	(1),(1,3),2)⊢	d,2
0(2(2	1),(1,3),2)⊢	d,6
0(2(2entier6	),(1,3),2)⊢	r,5
0(2(2A4	),(1,3),2)⊢	d,7
0(2(2A4)7	,(1,3),2)⊢	r,1
0(2L3	,(1,3),2)⊢	r,6
0(2A4	,(1,3),2)⊢	d,8
0(2A4,8	(1,3),2)⊢	d,2
0(2A4,8(2	1,3),2)⊢	d,6
0(2A4,8(2entier6	,3),2)⊢	r,5
0(2A4,8(2A4	,3),2)⊢	d,8
0(2A4,8(2A4,8	3),2)⊢	d,10
0(2A4,8(2A4,8entier10	),2)⊢	r,3
0(2A4,8(2A4	),2)⊢	d,7
0(2A4,8(2A4)7	,2)⊢	r,1
0(2A4,8L9	,2) ⊢	r,4
0(2A4	,2) ⊢	d,8
0(2A4,8	2) ⊢	d,10
0(2A4,8entier10	) ⊢	r,3
0(2A4	)⊢	d,7
0(2A4)7	⊢	r,1
0L1	⊢	accepter

Exercice 4 (Décoration de grammaire - 4 pts)

Nous souhaitons pouvoir créer un programme permettant de calculer et d'afficher des opérations entières ou réelles. La grammaire est la suivante :

$$\begin{aligned}
 L &\rightarrow E \text{'\backslash n'} L \\
 L &\rightarrow \epsilon \\
 E &\rightarrow E_1 + T \mid T \\
 T &\rightarrow T_1 * F \mid F \\
 F &\rightarrow \text{entier} \mid \text{reel} \mid (E)
 \end{aligned}$$

Nous considérons les fonctions suivantes :

```

/**
 * Affiche une valeur à l'écran.
 * @param valeur la valeur à afficher (entier ou réel)
 */
afficher(valeur)

/**
 * Convertit une valeur dans un autre type.
 * @param valeur la valeur à convertir
 * @param type INT pour convertir un réel en entier et DOUBLE pour l'inverse
 * @return la valeur
 */
convertir(valeur, type) : valeur

/**
 * Convertit une chaîne de caractères dans un type donné.
 * @param str la chaîne à convertir
 * @param type INT pour convertir en entier et DOUBLE pour convertir en réel
 * @return la valeur
 */
convstr(str, type) : valeur

```

1°) Quels sont les attributs nécessaires pour chaque symbole de la grammaire ? Précisez s'ils sont synthétisés ou hérités.

Solution : (1 pt) Nous pouvons spécifier un attribut synthétisé *lex* permettant de récupérer la valeur envoyée par l'analyseur lexical (pour entier et reel). Chaque symbole non terminal *L*, *E*, *T*, *F* possède les attributs synthétisés *valeur* et *type*.

2°) Proposez une décoration de la grammaire permettant d'afficher le résultat des opérations dès que l'utilisateur appuie sur la touche «entrée» (on ne gère pas les erreurs).

Solution : (3 pts)

```

L -> E '\n' L {
    afficher(E.valeur)
}
L -> ε { /* Rien à faire */ }
E -> E1 + T {
    Si E1.type = T.type Alors
        E.valeur = E1.valeur + T.valeur
        E.type = E1.type
    Sinon
        Si E1.type = INT Alors
            E.valeur = convertir(E1.valeur, DOUBLE) + T.valeur
        Sinon
            E.valeur = E1.valeur + convertir(T.valeur, DOUBLE)
        FinSi
        E.type = DOUBLE
    }

```

```
    FinSi
  }
E -> T {
  E.valeur = T.valeur
  E.type = T.type
}
T -> T1 * F {
  Si T1.type = F.type Alors
    T.valeur = T1.valeur * F.valeur
    T.type = T1.type
  Sinon
    Si T1.type = INT Alors
      T.valeur = convertir(T1.valeur, DOUBLE) * F.valeur
    Sinon
      T.valeur = T1.valeur * convertir(F.valeur, DOUBLE)
    FinSi
    T.type = DOUBLE
  FinSi
}
F -> entier {
  F.valeur = convstr(entier.lex, INT)
  F.type = INT
}
F -> reel {
  F.valeur = convstr(reel.lex, DOUBLE)
  F.type = REEL
}
F -> (E) {
  F.valeur = E.valeur
  F.type = E.type
}
```